

SQL Queries Guide

For Test Automation Engineers

From basics to advanced - with real query results

Sample Database Used in This Guide

users table

id	name
1	Jaime
2	Jon Snow
3	Theon Greyjoy

swords table

id	name	attack	owner_id
1	Oathkeeper	200	1
2	Wolf Bane	214	2
3	KrakenSlayer	185	3

1. Must-Know Queries (Daily Use)

SELECT with WHERE, AND/OR, IN, BETWEEN, LIKE

SELECT is the most fundamental SQL command. It retrieves data from tables. As a test automation engineer, you will use SELECT constantly to verify that your application stored or returned the correct data.

```
-- Basic SELECT: get all columns from users
SELECT * FROM users;
```

Result:

id	name
1	Jaime
2	Jon Snow
3	Theon Greyjoy

```
-- Select specific columns
SELECT name FROM users;
```

Result:

name
Jaime
Jon Snow
Theon Greyjoy

```
-- WHERE clause filters rows
SELECT * FROM users WHERE name = 'Jaime';
```

Result:

id	name
1	Jaime

```
-- IN checks against a list of values
SELECT * FROM users WHERE name IN ('Jaime', 'Jon Snow');
```

Result:

id	name
1	Jaime
2	Jon Snow

```
-- LIKE for pattern matching (% = any chars, _ = one char)
SELECT * FROM users WHERE name LIKE 'J%';
```

Result:

id	name
1	Jaime
2	Jon Snow

Other WHERE operators you should know:

AND / OR - combine conditions: WHERE age > 18 AND status = 'active'

BETWEEN - range filter: WHERE attack BETWEEN 185 AND 210

!= or <> - not equal: WHERE name != 'Jaime'

```
-- BETWEEN filters a range (inclusive)
SELECT * FROM swords WHERE attack BETWEEN 185 AND 210;
```

Result:

id	name	attack	owner_id
1	Oathkeeper	200	1
3	KrakenSlayer	185	3

JOIN - INNER, LEFT, RIGHT

JOINS combine rows from two or more tables based on a related column. Understanding JOINS is critical for verifying relationships between entities.

```
-- INNER JOIN: only matching rows from both tables
SELECT users.name, swords.name AS sword, swords.attack
FROM users
INNER JOIN swords ON users.id = swords.owner_id;
```

Result:

name	sword	attack
Jaime	Oathkeeper	200
Jon Snow	Wolf Bane	214
Theon Greyjoy	KrakenSlayer	185

```
-- LEFT JOIN: all rows from left table + matches from right
-- If a user had no sword, sword columns would show NULL
SELECT users.name, swords.name AS sword
FROM users
LEFT JOIN swords ON users.id = swords.owner_id;
```

Result:

name	sword
Jaime	Oathkeeper
Jon Snow	Wolf Bane
Theon Greyjoy	KrakenSlayer

RIGHT JOIN is the opposite of LEFT JOIN - it keeps all rows from the right table. In practice, LEFT JOIN is used far more often.

INSERT, UPDATE, DELETE

These commands modify data. In test automation, you use them for test data setup (INSERT), modifying state (UPDATE), and cleanup (DELETE).

```
-- INSERT a new row
INSERT INTO users (name) VALUES ('Arya Stark');
```

Result:

After INSERT, the users table now contains:

id	name
1	Jaime
2	Jon Snow
3	Theon Greyjoy
4	Arya Stark

```
-- UPDATE existing rows (always use WHERE!)
UPDATE users SET name = 'Jaime Lannister' WHERE id = 1;
```

Result:

After UPDATE:

id	name
1	Jaime Lannister
2	Jon Snow
3	Theon Greyjoy
4	Arya Stark

```
-- DELETE rows (always use WHERE!)  
DELETE FROM users WHERE id = 4;
```

Result:

After DELETE, Arya is removed. Table is back to original 3 rows.

WARNING: UPDATE and DELETE without WHERE will affect ALL rows in the table. Always double-check your WHERE clause.

ORDER BY, LIMIT, DISTINCT

These clauses help you sort, limit, and deduplicate results.

```
-- ORDER BY sorts results (ASC = ascending, DESC = descending)
SELECT * FROM users ORDER BY name ASC;
```

Result:

id	name
1	Jaime
2	Jon Snow
3	Theon Greyjoy

```
-- ORDER BY descending
SELECT * FROM swords ORDER BY attack DESC;
```

Result:

id	name	attack	owner_id
2	Wolf Bane	214	2
1	Oathkeeper	200	1
3	KrakenSlayer	185	3

```
-- LIMIT restricts number of rows returned
SELECT * FROM users LIMIT 2;
```

Result:

id	name
1	Jaime
2	Jon Snow

```
-- DISTINCT removes duplicate values
SELECT DISTINCT attack FROM swords;
```

Result:

attack
200
214
185

Aggregate Functions: COUNT, SUM, AVG, MIN, MAX

Aggregate functions calculate a single value from multiple rows. Essential for verifying data counts and totals in your tests.

```
-- COUNT: how many rows?
SELECT COUNT(*) FROM users;
```

Result:

count

3

```
-- SUM: total of a numeric column  
SELECT SUM(attack) FROM swords;
```

Result:

sum

599

```
-- AVG: average value  
SELECT AVG(attack) FROM swords;
```

Result:

avg

199.67

```
-- MIN / MAX: smallest / largest value  
SELECT MIN(attack), MAX(attack) FROM swords;
```

Result:

min

max

185

214

GROUP BY + HAVING

GROUP BY groups rows that share a value, then you can run aggregates on each group. HAVING filters groups (like WHERE but for aggregated results).

```
-- Count swords per owner
SELECT owner_id, COUNT(*) as sword_count
FROM swords
GROUP BY owner_id;
```

Result:

owner_id	sword_count
1	1
2	1
3	1

```
-- Find owners with attack power > 200
SELECT owner_id, SUM(attack) as total_attack
FROM swords
GROUP BY owner_id
HAVING SUM(attack) > 200;
```

Result:

owner_id	total_attack
2	214

NULL Handling: IS NULL, IS NOT NULL, COALESCE

NULL means 'no value' - not zero, not empty string. You cannot compare NULL with = or !=. You must use IS NULL / IS NOT NULL.

```
-- Assume we add a user with no sword:
-- INSERT INTO users (id, name) VALUES (4, 'Bran');
-- (Bran has no matching row in swords)

SELECT u.name, s.name AS sword
FROM users u
LEFT JOIN swords s ON u.id = s.owner_id
WHERE s.name IS NULL;
```

Result:

name	sword
Bran	NULL

```
-- COALESCE: return first non-NULL value (great for defaults)
SELECT u.name, COALESCE(s.name, 'No sword') AS weapon
FROM users u
LEFT JOIN swords s ON u.id = s.owner_id;
```

Result:

SQL for Test Automation Engineers

name	weapon
Jaime	Oathkeeper
Jon Snow	Wolf Bane
Theon Greyjoy	KrakenSlayer
Bran	No sword

2. Important Queries (Validation & Debugging)

Subqueries

A subquery is a query inside another query. Useful when you need to filter based on the results of another query.

```
-- Find users who own a sword
SELECT * FROM users
WHERE id IN (SELECT owner_id FROM swords);
```

Result:

id	name
1	Jaime
2	Jon Snow
3	Theon Greyjoy

```
-- Find users whose sword has attack > 200
SELECT * FROM users
WHERE id IN (
    SELECT owner_id FROM swords WHERE attack > 200
);
```

Result:

id	name
2	Jon Snow

EXISTS / NOT EXISTS

EXISTS checks if a subquery returns any rows. Often faster than IN for large datasets.

```
-- Find users who own a sword
SELECT * FROM users u
WHERE EXISTS (
    SELECT 1 FROM swords s WHERE s.owner_id = u.id
);
```

Result:

id	name
1	Jaime
2	Jon Snow
3	Theon Greyjoy

```
-- Find users who do NOT own a sword
-- (If Bran was added with id=4, he'd appear here)
SELECT * FROM users u
WHERE NOT EXISTS (
    SELECT 1 FROM swords s WHERE s.owner_id = u.id
);
```

Result:

(empty result - all users currently have swords)

UNION / UNION ALL

UNION combines results from two queries. UNION removes duplicates, UNION ALL keeps them (faster).

```
-- Combine user names and sword names into one list
SELECT name FROM users
UNION ALL
SELECT name FROM swords;
```

Result:

name
Jaime
Jon Snow
Theon Greyjoy
Oathkeeper
Wolf Bane
KrakenSlayer

CASE WHEN

CASE WHEN adds conditional logic inside a query - like an if/else in SQL.

```
SELECT name, attack,
CASE
  WHEN attack >= 210 THEN 'Legendary'
  WHEN attack >= 195 THEN 'Epic'
  ELSE 'Common'
END AS rarity
FROM swords;
```

Result:

name	attack	rarity
Oathkeeper	200	Epic
Wolf Bane	214	Legendary
KrakenSlayer	185	Common

Aliases (AS)

Aliases give temporary names to columns or tables, making queries more readable.

```
-- Column alias
SELECT COUNT(*) AS total_users FROM users;
```

Result:

total_users
3

```
-- Table alias (essential for JOINS)
SELECT u.name, s.name AS sword, s.attack
FROM users AS u
JOIN swords AS s ON u.id = s.owner_id;
```

Result:

name	sword	attack
Jaime	Oathkeeper	200
Jon Snow	Wolf Bane	214
Theon Greyjoy	KrakenSlayer	185

Transactions: BEGIN, COMMIT, ROLLBACK

Transactions group multiple operations into one atomic unit. Critical for test isolation - you can set up data, run your test, then ROLLBACK to undo everything.

```
-- Basic transaction
BEGIN;
INSERT INTO users (name) VALUES ('Test User');
INSERT INTO swords (name, attack, owner_id)
VALUES ('Test Sword', 100, 4);
```

```
COMMIT;  -- saves both changes

-- Rollback: undo everything if something goes wrong
BEGIN;
  DELETE FROM swords WHERE owner_id = 1;
  -- Oops, wrong user!
ROLLBACK;  -- nothing was deleted
```

Test automation tip: Wrap your test data setup in a transaction. After assertions, ROLLBACK to leave the database clean for the next test.

3. Good to Know (Advanced Verification)

Window Functions: ROW_NUMBER(), RANK()

Window functions perform calculations across rows related to the current row without collapsing them into groups (unlike GROUP BY).

```
-- Rank swords by attack power
SELECT name, attack,
       ROW_NUMBER() OVER (ORDER BY attack DESC) as rank
FROM swords;
```

Result:

name	attack	rank
Wolf Bane	214	1
Oathkeeper	200	2
KrakenSlayer	185	3

CTEs (Common Table Expressions)

CTEs let you define temporary named result sets using WITH. They make complex queries much more readable.

```
-- Find users with powerful swords (attack > 190)
WITH powerful_swords AS (
  SELECT owner_id, name AS sword, attack
  FROM swords
  WHERE attack > 190
)
SELECT u.name, ps.sword, ps.attack
FROM users u
JOIN powerful_swords ps ON u.id = ps.owner_id;
```

Result:

name	sword	attack
Jaime	Oathkeeper	200
Jon Snow	Wolf Bane	214

EXPLAIN / EXPLAIN ANALYZE

EXPLAIN shows how the database plans to execute your query. EXPLAIN ANALYZE actually runs it and shows real timing. Useful when debugging slow test suites caused by slow queries.

```
-- See the query plan (does NOT execute the query)
EXPLAIN SELECT * FROM users WHERE name = 'Jaime';

-- See plan + actual execution time
EXPLAIN ANALYZE SELECT * FROM users WHERE name = 'Jaime';
```

Result:

Returns a query plan showing: Seq Scan on users, cost, rows, width, and actual time/loops if using ANALYZE. Look for 'Seq Scan' (slow on big tables) vs 'Index Scan' (fast).

CREATE, DROP, ALTER TABLE

Schema commands define the structure of your database. Understanding them helps you read migration files and set up test databases.

```
-- Create a table
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL
);

-- Create swords table with foreign key
CREATE TABLE swords (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  attack INTEGER NOT NULL,
  owner_id INTEGER REFERENCES users(id)
);

-- Add a column
ALTER TABLE users ADD COLUMN email TEXT;

-- Drop (delete) a table
DROP TABLE IF EXISTS swords;
```

Constraints: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL

Constraints enforce rules on your data. Knowing them helps you write tests that verify data integrity.

```
-- PRIMARY KEY: unique identifier for each row (id in users)
-- FOREIGN KEY: links to another table (owner_id -> users.id)
-- UNIQUE: no duplicate values allowed
-- NOT NULL: value is required

-- Example: try inserting a sword with invalid owner_id
INSERT INTO swords (name, attack, owner_id)
VALUES ('Ghost Blade', 300, 999);
-- ERROR: insert violates foreign key constraint
-- (user with id=999 does not exist)
```

4. Test Automation Specific Patterns

Verify data after an API call

After your test calls POST /api/users to create 'Arya', verify it was saved:

```
SELECT * FROM users WHERE name = 'Arya';
```

Result:

id	name
4	Arya

Set up test data

Insert known data before your test runs:

```
INSERT INTO users (name) VALUES ('Test User')  
RETURNING id; -- get the generated ID back
```

Result:

id
4

Clean up after tests

Remove test data to keep the database clean:

```
-- Delete specific test data  
DELETE FROM swords WHERE owner_id = 4;  
DELETE FROM users WHERE name = 'Test User';  
  
-- Or use TRUNCATE to remove ALL rows (faster, resets IDs)  
TRUNCATE TABLE swords CASCADE;
```

Validate no duplicates exist

Check for data integrity issues after a test run:

```
SELECT name, COUNT(*) as count  
FROM users  
GROUP BY name  
HAVING COUNT(*) > 1;
```

Result:

(empty result - no duplicates found)

If this returns rows, you have duplicate data that needs investigation.

Check data integrity (find orphaned records)

Find swords that reference non-existent users:


```
SELECT s.*
FROM swords s
LEFT JOIN users u ON s.owner_id = u.id
WHERE u.id IS NULL;
```

Result:

(empty result - no orphaned swords found)

Compare before/after state

Count records before and after a test action:

```
-- Before test: 3 users
SELECT COUNT(*) as user_count FROM users;
```

Result:

user_count

3

```
-- After test inserts a user: 4 users
SELECT COUNT(*) as user_count FROM users;
```

Result:

user_count

4

5. Using SQL in Python (psycopg2)

In test automation with Python, you typically use a library like psycopg2 to execute SQL queries. Here are common patterns:

Basic query with parameterization

```
import psycopg2

conn = psycopg2.connect(DATABASE_URL)
with conn.cursor() as cur:
    # SAFE: parameterized query
    cur.execute(
        "SELECT * FROM users WHERE name = %s;",
        ("Jaime",)
    )
    result = cur.fetchone()
    # result = (1, 'Jaime')

    # NEVER do this (SQL injection risk):
    # cur.execute(f"SELECT * FROM users WHERE name = '{name}'")
```

Fetch methods

```
cur.execute("SELECT * FROM users;")

# fetchone() - returns single row as tuple
row = cur.fetchone()
# row = (1, 'Jaime')

# fetchall() - returns all rows as list of tuples
rows = cur.fetchall()
# rows = [(1, 'Jaime'), (2, 'Jon Snow'), (3, 'Theon Greyjoy')]

# fetchmany(n) - returns n rows
some = cur.fetchmany(2)
# some = [(1, 'Jaime'), (2, 'Jon Snow')]
```

Assert query results in tests

```
def test_user_has_sword(db_cursor):
    db_cursor.execute(
        "SELECT u.name, s.name FROM users u "
        "JOIN swords s ON u.id = s.owner_id "
        "WHERE u.name = %s",
        ("Jaime",)
    )
    result = db_cursor.fetchone()
    # result = ('Jaime', 'Oathkeeper')
```

```
assert result is not None, "User not found"
assert result[0] == "Jaime"
assert result[1] == "Oathkeeper"
```

Transaction rollback for test cleanup

```
import pytest

@pytest.fixture
def db_conn():
    conn = psycopg2.connect(DATABASE_URL)
    yield conn
    conn.rollback() # undo all changes after each test
    conn.close()

def test_insert_user(db_conn):
    with db_conn.cursor() as cur:
        cur.execute("INSERT INTO users (name) VALUES (%s)", ("Test",))
        cur.execute("SELECT COUNT(*) FROM users")
        assert cur.fetchone()[0] == 4 # original 3 + 1
    # After test: db_conn.rollback() runs automatically
    # The inserted user is removed - database stays clean!
```

Quick Reference Cheat Sheet

Command	Description
SELECT	Retrieve data from table(s)
WHERE	Filter rows by condition
JOIN	Combine rows from multiple tables
INSERT INTO	Add new rows
UPDATE ... SET	Modify existing rows
DELETE FROM	Remove rows
ORDER BY	Sort results
LIMIT	Restrict number of results
DISTINCT	Remove duplicate values
GROUP BY	Group rows for aggregation
HAVING	Filter groups (after GROUP BY)
COUNT/SUM/AVG	Aggregate functions
IS NULL	Check for missing values
COALESCE	Return first non-NULL value
IN / NOT IN	Match against a list
BETWEEN	Match within a range
LIKE	Pattern matching (% and _)
EXISTS	Check if subquery has results
UNION	Combine two result sets
CASE WHEN	Conditional logic
BEGIN/COMMIT	Transaction control
ROLLBACK	Undo transaction changes
EXPLAIN ANALYZE	Query performance analysis
ROW_NUMBER()	Window function - row numbering
WITH ... AS	CTE - named subquery